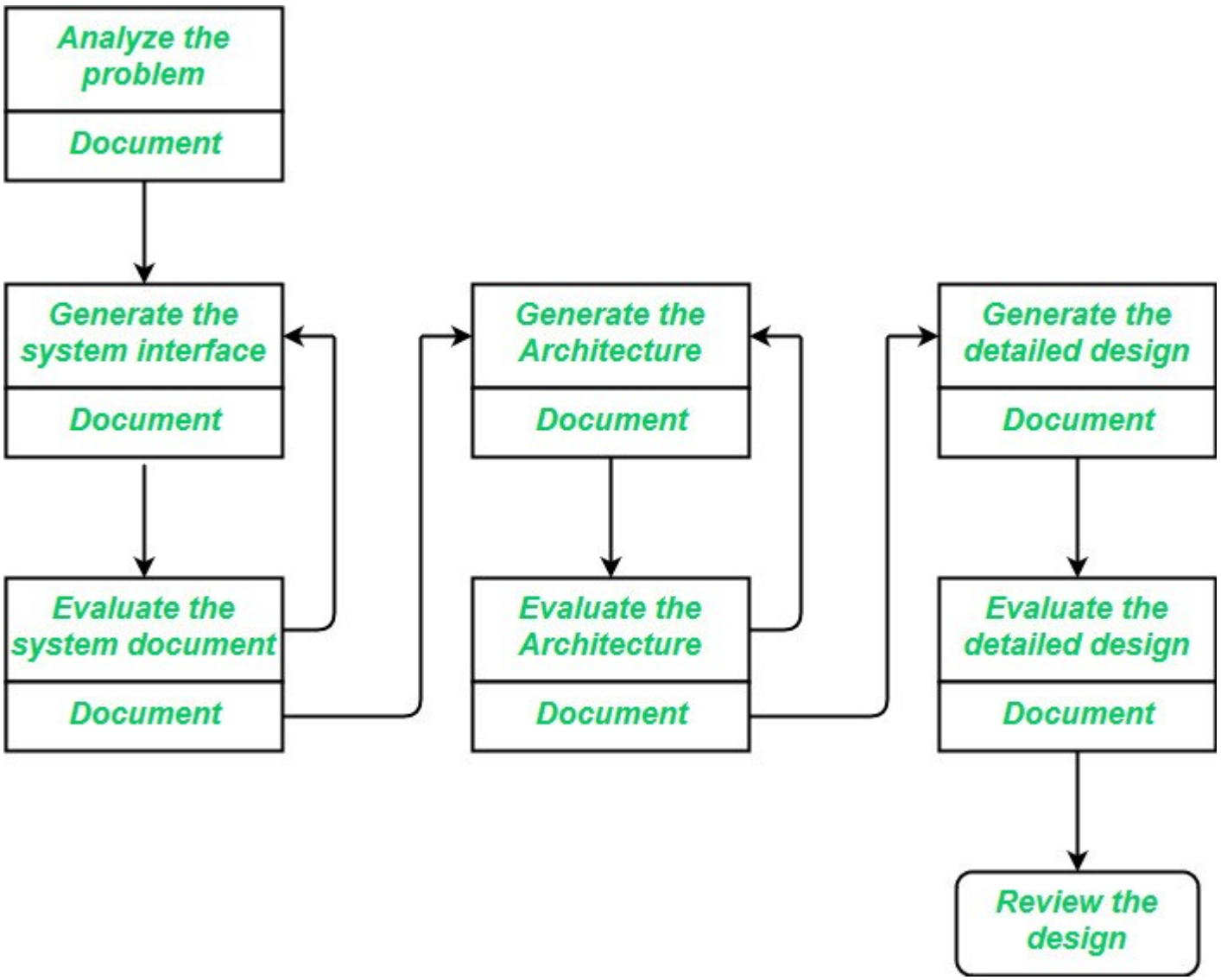
Software Design Process

The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language. The software design process can be divided into the following three levels or phases of design:

1. Interface Design
2. Architectural Design
3. Detailed Design

Elements of a System

1. **Architecture:** This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture.
2. **Modules:** These are components that handle one specific task in a system. A combination of the modules makes up the system.
3. **Components:** This provides a particular function or group of related functions. They are made up of modules.
4. **Interfaces:** This is the shared boundary across which the components of a system exchange information and relate.
5. **Data:** This is the management of the information and data flow.



Software Design Process

Interface Design

Interface design is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a black box. Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called agents.

Interface design should include the following details:

1. Precise description of events in the environment, or messages from agents to which the system must respond.
2. Precise description of the events or messages that the system must produce.
3. Specification of the data, and the formats of the data coming into and going out of the system.

4. Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

Architectural Design

Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored. Issues in architectural design includes:

1. Gross decomposition of the systems into major components.
2. Allocation of functional responsibilities to components.
3. Component Interfaces.
4. Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
5. Communication and interaction between components.

The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

Detailed Design

Detailed design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures. The detailed design may include:

1. Decomposition of major system components into program units.
2. Allocation of functional responsibilities to units.
3. User interfaces.
4. Unit states and state changes.
5. Data and control interaction between units.
6. Data packaging and implementation, including issues of scope and visibility of program elements.
7. Algorithms and data structures.

Design engineering encompasses the set of principles, concepts, and practices that lead to the development of a high- quality system or product.

✓ Design principles establish an overriding philosophy that guides the designer in the work that is performed.

✓ Design concepts must be understood before the mechanics of design practice are applied and

✓ Design practice itself leads to the creation of various representations of the software that serve as a guide for the construction activity that follows.

What is design?

Design is what virtually every engineer wants to do. It is the place where creativity rules – customer's requirements, business needs, and technical considerations all come together in the formulation of a product or a system. Design creates a representation or model of the software, but unlike the analysis model, the design model provides detail about software data structures, architecture, interfaces, and components that are necessary to implement the system.

Why is it important?

Design allows a software engineer to model the system or product that is to be built. This model can be assessed for quality and improved before code is generated, tests are conducted, and end – users become involved in large numbers. Design is the place where software quality is established.

The goal of design engineering is to produce a model or representation that exhibits firmness, commodity, and delight. To accomplish this, a designer must practice diversification and then convergence. Another goal of software design is to derive an architectural rendering of a system. The rendering serves as a framework from which more detailed design activities are conducted.

1) DESIGN PROCESS AND DESIGN QUALITY:

Software design is an iterative process through which requirements are translated into a "blueprint" for constructing the software.

Goals of design:

McGlaughlin suggests three characteristics that serve as a guide for the evaluation of a good design.

➢ The design must implement all ofthe explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.

➢ The design must be a readable, understandable guide for those who generate code and forthose who test and subsequently support the software.

➢ The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Quality guidelines:

In order to evaluate the quality of a design representation we must establish technical criteria for good design. These are the following guidelines:

1. A design should exhibit an architecture that

a. has been created using recognizable architectural styles or patterns

b. is composed of components that exhibit good design characteristics and

c. can be implemented in an evolutionary fashion, thereby facilitating implementation and testing.

2. A design should be modular; that is, the software should be logically partitioned into elements or subsystems.

3. A design should contain distinct representation of data, architecture, interfaces andcomponents.

4. A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

5. A design should lead to components that exhibit independent functionalcharacteristics.

6. A design should lead to interface that reduce the complexity of connections between components and with the external environment.

7. A design should be derived using a repeatable method that is driven byinformation obtained during software requirements analysis.

8. A design should be represented using a notation that effectively communication its meaning.

These design guidelines are not achieved by chance. Design engineering encourages good design through the application of fundamental design principles, systematic methodology, and thorough review.

Quality attributes:

The FURPS quality attributes represent a target for all software design:

➢ **Functionality** is assessed by evaluating the feature set and capabilities ofthe program, the generality of the functions that are delivered, and the security of the overall system.

➢ **Usability** is assessed by considering human factors, overall aesthetics, consistencyand

documentation.

➢ **Reliability** is evaluated by measuring the frequency and severity of failure, the accuracy of output results, and the mean – time –to- failure (MTTF), the ability to recover from failure, and the predictability of the program.

➢ **Performance** is measured by processing speed, response time, resource consumption, throughput, and efficiency

➢ **Supportability** combines the ability to extend the program (extensibility), adaptability, serviceability-these three attributes represent a more common termmaintainability

Not every software quality attribute is weighted equally as the software design is developed. One application may stress functionality with a special emphasis on security. Another may demand performance with particular emphasis on processing speed. A third might focus on reliability.